



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-10722 cilium/ebpf malformed BTF parsing denial-of-service risk

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-07
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-10722 cilium/ebpf malformed BTF parsing denial-of-service risk - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-07 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-10722 cilium/ebpf malformed BTF parsing denial-of-service risk

1. Executive Summary

CVE-2026-10722 affects the Go package `github.com/cilium/ebpf` through version `v0.21.0` and is reachable through public parser entry points such as `LoadCollectionSpec` and `LoadCollectionSpecFromReader`. The collected primary sources show that malformed `.BTF` / `.BTF.ext` content can drive parser failures in the same loader path, including an integer-overflow bounds-check defect in `btf/btf.go` that upstream fixed by changing `uint32` addition to `uint64` addition.

For defenders, the practical risk is process-level denial of service in software that parses attacker-controlled or less-trusted eBPF ELF/BTF artifacts. This is not a kernel privilege-escalation issue and the gathered evidence does not support remote code execution. Exposure is concentrated in build systems, security tooling, controllers, or ingestion services that accept external eBPF objects and parse them inside long-running Go processes.

2. Vulnerability Metadata

Field	Value
CVE	<code>CVE-2026-10722</code>
Product	<code>github.com/cilium/ebpf</code>
Vendor / maintainer	Cilium
Affected component	<code>LoadCollectionSpec</code> / <code>LoadCollectionSpecFromReader</code> loader path, including <code>btf/btf.go</code> <code>loadRawSpec</code>
Affected versions in the CVE record	<code>0.1</code> through <code>0.21.0</code>
Weaknesses listed in the CVE record	<code>CWE-190 Integer Overflow</code> , <code>CWE-189 Numeric Error</code>
CVSS v3.1 in the CVE record	<code>3.3 / LOW</code>
CVSS v4.0 in the CVE record	<code>4.8 / MEDIUM</code>
Upstream patch commit named in the advisory	<code>533dfc82fd228bfadf42ea7180c39de7d9af47fa</code>
Upstream hardening PR	<code>cilium/ebpf#2021</code>

3. Source Review & Confidence

Primary evidence is strong that the affected parser path existed and was fixed upstream:

Source	Contribution	Confidence
Reviewed source	Syndicated listing for CVE-2026-10722 and initial intake source.	Useful as a routing source, but not detailed enough to stand alone for technical conclusions.
Reviewed source	Official CVE JSON 5 record from VulDB; affected versions run through <code>0.21.0</code> ; weaknesses listed as <code>CWE-190</code> and <code>CWE-1</code>	Primary source for normalized CVE metadata.
Reviewed source	GitHub advisory for <code>github.com/cilium/ebpf</code> ; repeats affected	Strong corroborating advisory source

	version ceiling `0.21.0`; describes manipulation of parse	aligned to the Go ecosystem.
Reviewed source	Upstream issue reports malformed `.BTF` / `.BTF.ext` parser panic reachable through `LoadCollectionSpec` and `LoadCollec	Primary technical source for attack preconditions and practical impact.
Reviewed source	Upstream hardening PR merged on 2026-05-27; fixes multiple malformed-input parser panic conditions in the same BTF load	Strong source for remediation context and patch grouping.
Reviewed source	Changes `btf/btf.go` bounds validation from `int(...)` addition to `uint64(...)` addition for string and type section ch	Best source for the exact code-level fix named in the CVE record.
Reviewed source	Public Go module version listing still tops out at `v0.21.0` at review time.	Useful for assessing upgrade availability; does not by itself prove vulnerability details.
Reviewed source	Public PoC pack documents malformed BTF string-table parsing leading to panic and process-level denial of service.	Valuable exploitation context, but secondary to the upstream issue/PR and not used as a sole source

Analyst confidence is medium rather than high because the public issue/PoC material emphasizes malformed string-table panic behavior while the CVE text and named patch focus on integer overflow in length checking. Those flaws sit in the same BTF parsing path and were fixed together in the same upstream hardening effort, but the public wording is not perfectly aligned across sources.

4. Vulnerability Details

The clearest advisory-backed defect is an integer-overflow risk in `btf/btf.go` `loadRawSpec` while validating BTF string and type section bounds. Prior to the fix, the code performed checks equivalent to:

- `int(header.StringOff + header.StringLen) > len(btf)`
- `int(header.TypeOff + header.TypeLen) > len(btf)`

Because `StringOff`, `StringLen`, `TypeOff`, and `TypeLen` are `uint32` values derived from parsed input, an attacker-controlled malformed BTF blob could trigger wraparound before the result was converted for comparison. Upstream fixed this by widening the addition to `uint64`, preventing overflow during bounds validation.

The surrounding upstream PR also hardened adjacent malformed-input paths in BTF parsing:

- `a385fe229c38ded79ad67bfa21f8da07d72eecc5` returns an error instead of panicking when a malformed string table lacks a terminating NUL byte.
- `c8c6a424703dae02274386e396ad83f5800f65d7` adds out-of-bounds checks for BTF header remainder parsing.
- `533dfc82fd228bfadf42ea7180c39de7d9af47fa` fixes integer-overflow-prone length checks in `loadRawSpec`.

Taken together, the evidence supports treating CVE-2026-10722 as a malformed-BTF parser robustness issue in `cilium/ebpf` that can terminate a parsing process when hostile artifact input reaches the vulnerable code path.

5. Attack Scenarios

1. A build or CI pipeline accepts externally supplied eBPF ELF artifacts for testing or packaging and uses `LoadCollectionSpec` or `LoadCollectionSpecFromReader` during validation.
2. A security or platform engineering tool ingests customer-provided or third-party eBPF programs to inspect metadata before deployment.
3. A multi-tenant controller or service parses uploaded BTF-containing objects inside a long-running worker process.

In each case, the attacker does not need kernel execution. They need a path to supply crafted local artifact content to a Go application that uses the vulnerable parser path.

6. Threat Landscape & Exploitation Status

- Public disclosure exists.
- A public GitHub issue and public proof-of-concept gist were referenced in the CVE record.
- The strongest demonstrated impact in the gathered material is denial of service against the parsing process.
- No evidence was collected in this task showing in-the-wild exploitation, CISA KEV inclusion, or ransomware/operator tradecraft tied to this CVE.
- No evidence was collected showing a fixed tagged module release newer than `v0.21.0` as of 2026-06-08; the public Go module version list still tops out at `v0.21.0`.

7. Affected Products and Exposure

Affected exposure is narrower than a typical remote service vulnerability:

- Directly affected: applications or internal tooling that vendor `github.com/cilium/ebpf` at versions up to `v0.21.0` and parse less-trusted ELF/BTF content.
- Potentially affected environments: CI/CD systems, artifact scanners, eBPF packaging workflows, admission or orchestration helpers, and internal developer tooling.
- Not supported by the gathered evidence: kernel compromise, network-only exploitation with no artifact-ingestion step, or generic exposure for every Cilium deployment.

Organizations should prioritize environments where untrusted users, tenants, repositories, or external feeds can influence which eBPF artifacts are parsed.

8. Mitigation and Workarounds

Priority mitigation steps:

4. Move to the first upstream release that includes PR `#2021` once a tagged version is available.
5. If a fixed release is not yet available in your dependency channel, vendor or cherry-pick the upstream hardening commits into internal builds, especially `533dfc82fd228bfadf42ea7180c39de7d9af47fa`.
6. Treat eBPF ELF/BTF inputs as untrusted content and avoid parsing them inside high-value, long-lived worker processes without isolation.
7. Add crash containment around parsing workflows where feasible, such as subprocess isolation, bounded worker restarts, and panic monitoring.
8. Inventory internal projects importing `github.com/cilium/ebpf` and confirm whether they parse externally sourced artifacts or only trusted internally built objects.

9. Detection Engineering

This issue does not produce strong network-native signatures. Detection should focus on software inventory and application telemetry:

Signal	Telemetry Source	Analytic Goal
Identify `github.com/cilium/ebpf` dependencies at `v0.21.0` or earlier in `go.mod`, vendored source, SBOMs, and container images.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Alert on application crashes or panic traces referencing `LoadCollectionSpec`, `LoadCollectionSpecFromReader`,	Application logs, service metrics, EDR / crash telemetry	Spot exploitation symptoms or instability

`btf/btf.go`, or BTF string-table parsing.		
Monitor artifact-ingestion workflows for failed parses of unusual or externally supplied eBPF ELF inputs.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Flag internal tooling that accepts tenant, customer, or internet-sourced BPF objects without strong provenance controls.	Network, auth, proxy, or service-access logs	Identify externally reachable or suspiciously accessed systems

9.1 Detection Query

```
// Build and package provenance pivots for CVE-2026-10722 cilium/ebpf malformed BTF parsing denial-of-
service risk
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "go env", "pip show", "pip list", "syft", "grype",
"sbom")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
| order by Timestamp desc
```

9.2 Exposure / Triage Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

10. Threat Hunting

Suggested hunt questions:

- Which repositories and deployed binaries import `github.com/cilium/ebpf`, and what versions are in use?
- Which jobs, services, or controllers parse `.o`, `.elf`, `.BTF`, or `.BTF.ext` content supplied from outside the trust boundary?
- Are there recent crash logs or restart spikes in parser-adjacent workers around BTF parsing functions?
- Have any suspicious eBPF artifacts been uploaded, mirrored, or scanned shortly before parser crashes?

10.1 Hunt Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

11. MITRE ATT&CK Mapping

No ATT&CK technique is assigned with confidence for this report. The collected evidence supports a parser denial-of-service condition caused by malicious artifact content, but it does not document a mature intrusion chain, malware family, or operator tradecraft pattern that would justify a precise ATT&CK mapping.

12. Validation / Lab Testing

This task validated the report against collected upstream and advisory material rather than running a fresh local exploit reproduction. Analyst verification steps completed:

- Confirmed the public CVE JSON 5 record and GitHub advisory.
- Retrieved the upstream issue, PR, and commit metadata for the cited hardening work.
- Confirmed the named fix commit modifies `btf/btf.go` bounds checks with `uint64` arithmetic.
- Confirmed the public Go module version list still exposes `v0.21.0` as the latest tagged version available via the module proxy at review time.

A referenced public PoC exists, but no Lost Boys Cyber lab execution was performed as part of this intake review.

13. Recommended Actions Summary

Priority	Owner	Action
High	Security / Platform Owners	Inventory all internal uses of `github.com/cilium/ebpf` and identify any parser exposure to less-trusted BPF artifacts.
Critical	Platform / Application Owners	Patch or vendor the upstream hardening commits immediately if exposed workflows exist.
High	Security / Platform Owners	Prefer upgrading to the first tagged release that contains PR `#2021` once available.
High	SOC / Detection Engineering	Add crash monitoring and containment around artifact-parsing workers.
High	Security / Platform Owners	Record this CVE as a low-severity but operationally relevant parser DoS issue for teams handling untrusted eBPF content.

14. References

9. Microsoft Security Response Center, `CVE-2026-10722`
10. CVE Project JSON 5 record for `CVE-2026-10722`
11. GitHub Advisory `GHSA-xhgw-qwwf-pg32`
12. `cilium/ebpf` issue `#2019`, `BTF string offset boundary check can panic parser on malformed ELF/BTF input`
13. `cilium/ebpf` pull request `#2021`, `BTF: fixed panics during parsing of malformed input`
14. `cilium/ebpf` commit `533dfc82fd228bfadf42ea7180c39de7d9af47fa`
15. Go module proxy version listing for `github.com/cilium/ebpf`